



ROBERT v 1.2.1 2025/03/19 19:30:06

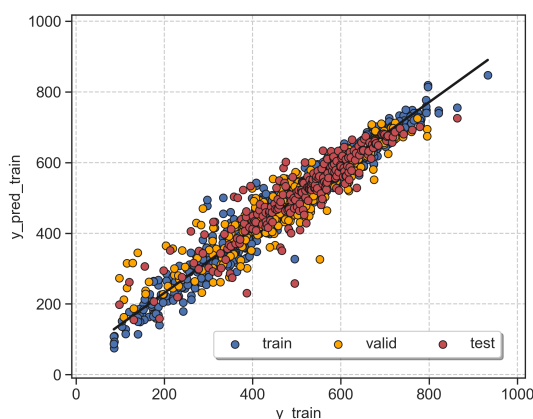
How to cite: Dalmau, D.; Alegre Requena, J. V. WIREs Comput Mol Sci. 2024, DOI: 10.1002/WCMS.1733

**Section A. ROBERT Score***This score is designed to evaluate the models using different metrics.***No PFI (standard descriptor filter):**

Model = GB · Train:Validation:Test = 72:18:10

Points(train+valid.):descriptors = 2119:16

Score = 7 / 10

**MODERATE**

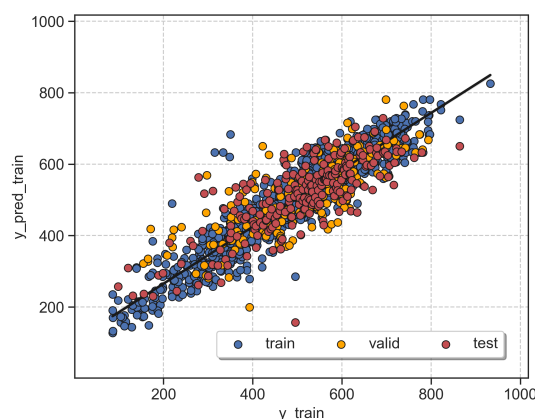
Train :  $R^2 = 0.95$ , MAE =  $2.1e+01$ , RMSE =  $2.9e+01$   
 Valid. :  $R^2 = 0.88$ , MAE =  $3.4e+01$ , RMSE =  $4.8e+01$   
 Test :  $R^2 = 0.84$ , MAE =  $3.8e+01$ , RMSE =  $5.3e+01$

**PFI (only most important descriptors):**

Model = RF · Train:Validation:Test = 81:9:10

Points(train+valid.):descriptors = 2119:12

Score = 4 / 10

**WEAK**

Train :  $R^2 = 0.89$ , MAE =  $3.3e+01$ , RMSE =  $4.6e+01$   
 Valid. :  $R^2 = 0.6$ , MAE =  $6.3e+01$ , RMSE =  $8.2e+01$   
 Test :  $R^2 = 0.63$ , MAE =  $6e+01$ , RMSE =  $8e+01$

**Severe warnings**

- ☒ No severe warnings detected

**Moderate warnings**

- ☒ Uneven y distribution (Section C)
- ☒ Potential "faulty" outliers (Section E)

**Overall assessment**

- ☒ Decent model, but it has limitations

**Severe warnings**

- ☒ No severe warnings detected

**Moderate warnings**

- ☒ Some tests are unclear (Section B.1)
- ☒ Uneven y distribution (Section C)
- ☒ Potential "faulty" outliers (Section E)

**Overall assessment**

- ☒ The model is unreliable



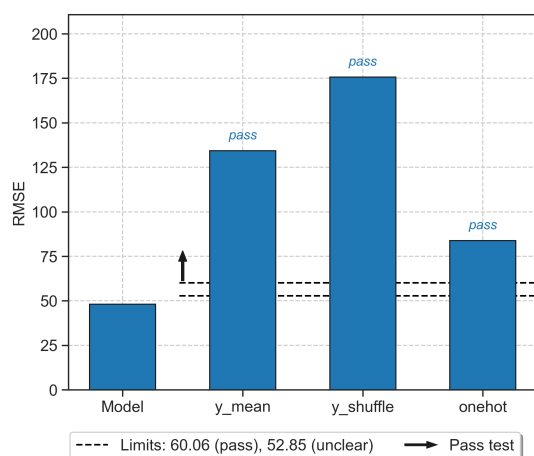
## Section B. Advanced Score Analysis

This section explains each component that comprises the ROBERT score.

### 1. Model vs "flawed" models (3 / 3 )

The model predicts right for the right reasons.

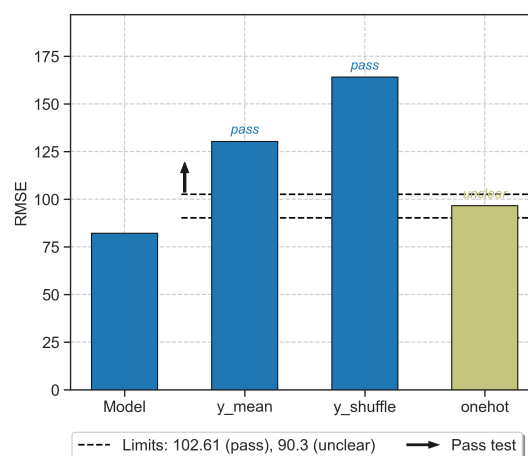
Pass: +1, Unclear: 0, Fail: -1. [Details here.](#)



### 1. Model vs "flawed" models (2 / 3 )

WARNING! The model might have important flaws.

Pass: +1, Unclear: 0, Fail: -1. [Details here.](#)



### 2. Predictive ability of the model (1 / 2 )

Moderate predictive ability with  $R^2$  (test) = 0.84.

$R^2$  0.70-0.85: +1,  $R^2$  >0.85: +2.

### 2. Predictive ability of the model (0 / 2 )

Low predictive ability with  $R^2$  (test) = 0.63.

$R^2$  0.70-0.85: +1,  $R^2$  >0.85: +2.

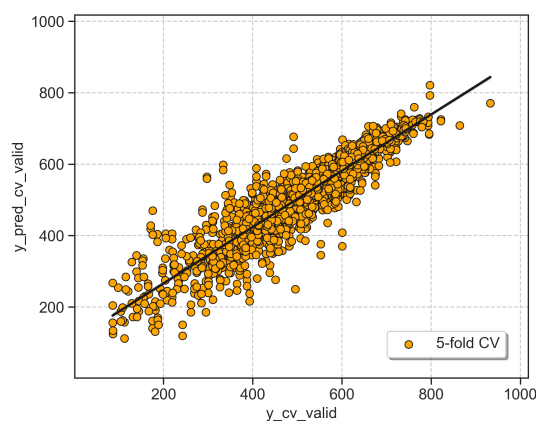
### 3. Cross-validation (5-fold CV) of the model

Overfitting analysis on the model with 3a and 3b:

#### 3a. CV predictions train + valid. (1 / 2 )

Moderate predictive ability with  $R^2$  (5-fold CV) = 0.84.

$R^2$  0.70-0.85: +1,  $R^2$  >0.85: +2.



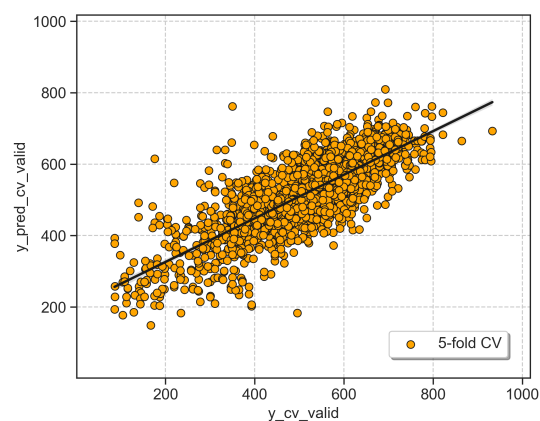
### 3. Cross-validation (5-fold CV) of the model

Overfitting analysis on the model with 3a and 3b:

#### 3a. CV predictions train + valid. (0 / 2 )

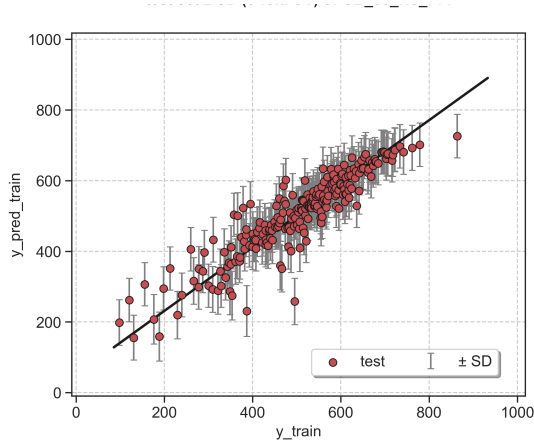
Low predictive ability with  $R^2$  (5-fold CV) = 0.62.

$R^2$  0.70-0.85: +1,  $R^2$  >0.85: +2.



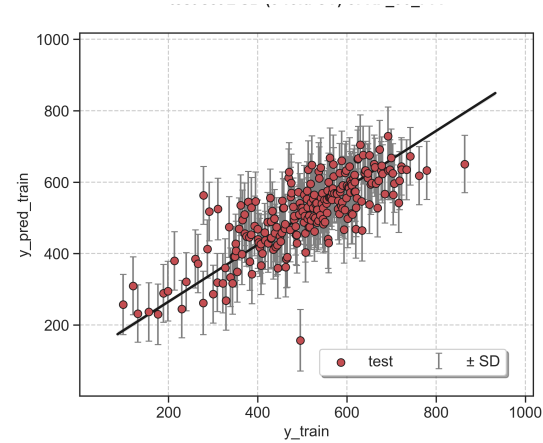
3b. Avg. standard deviation (SD) (1 / 2 )

Moderate variation, 4\*SD (test) = 248.9 (29% y-range).  
4\*SD 25-50% y-range: +1, 4\*SD < 25% y-range: +2.  
[Details here.](#)



3b. Avg. standard deviation (SD) (1 / 2 )

Moderate variation, 4\*SD (test) = 329.7 (39% y-range).  
4\*SD 25-50% y-range: +1, 4\*SD < 25% y-range: +2.  
[Details here.](#)



4. Points(train+valid.):descriptors (1 / 1 )

Decent number of descps. (ratio 2119:16).  
5 or more points per descriptor: +1.

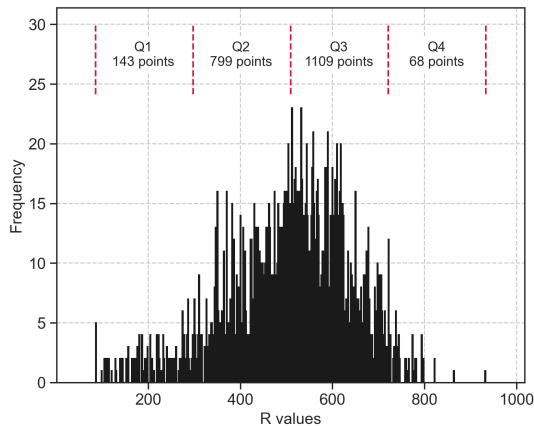
4. Points(train+valid.):descriptors (1 / 1 )

Decent number of descps. (ratio 2119:12).  
5 or more points per descriptor: +1.



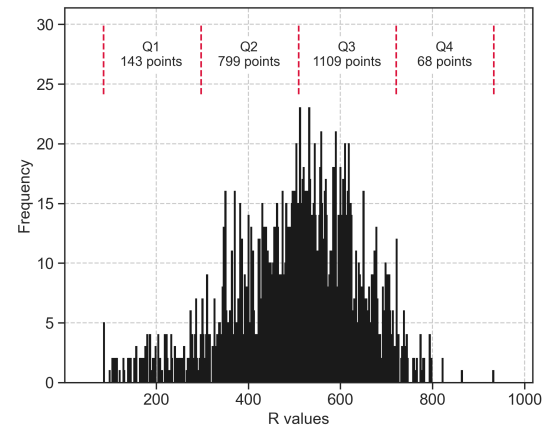
Section C. Distribution of y Values

This section shows the distribution of y values within the training and validation sets.



y distribution analysis

x WARNING! Your data is not uniform (Q4 has 68 points while Q3 has 1109)



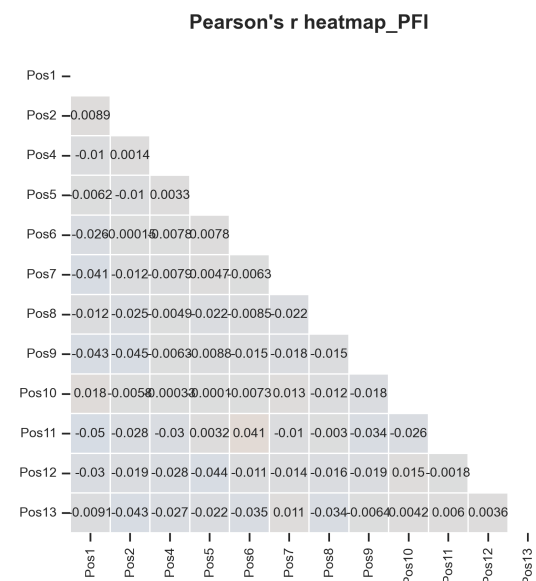
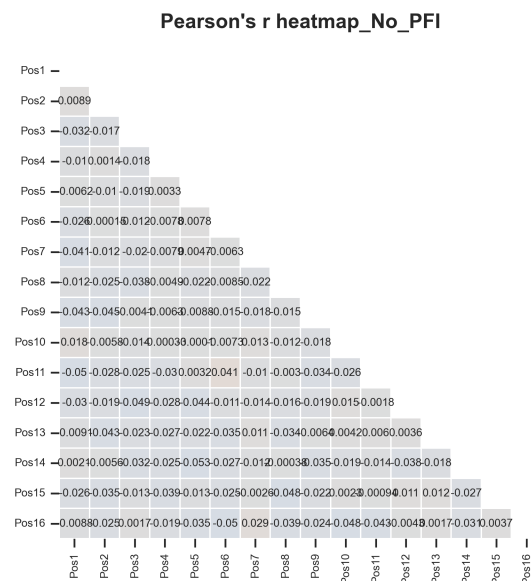
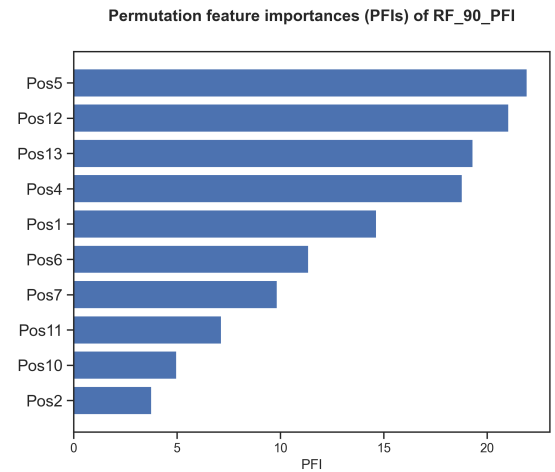
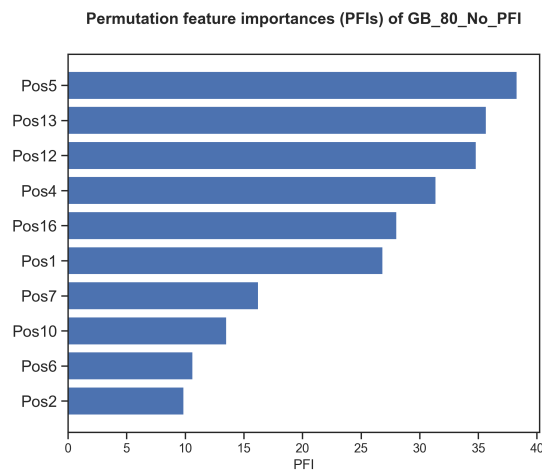
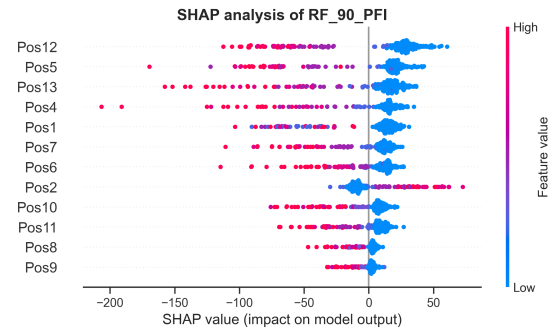
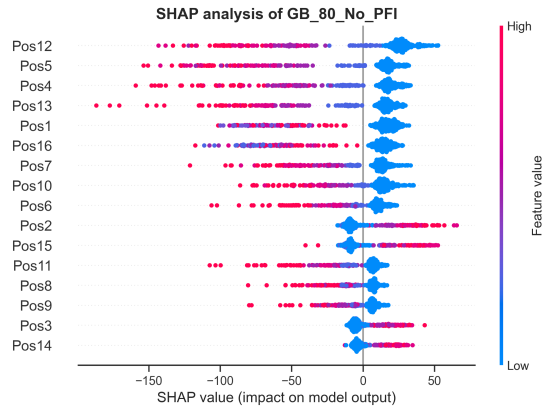
y distribution analysis

x WARNING! Your data is not uniform (Q4 has 68 points while Q3 has 1109)



## Section D. Feature Importances

This section presents feature importances measured using the validation set.



### Correlation analysis

- o Correlations between variables are acceptable

### Correlation analysis

- o Correlations between variables are acceptable





## Section E. Outlier Analysis

*This section detects outliers using the standard deviation (SD) of errors from the training set.*

**No PFI (standard descriptor filter):**Outliers (max. 10 shown)

Train: 66 outliers out of 1695 datapoints (3.9%)

- 6Heliceno\_2-brom\_3-brom\_5-fluor\_12-fluor\_14-brom\_15-brom.log (3.3 SDs)
- 6Heliceno\_2-fluor\_11-chlor\_12-chlor\_13-iod\_15-brom.log (2.1 SDs)
- 6Heliceno\_3-fluor\_4-iod\_11-fluor\_15-fluor.log (2.1 SDs)
- 6Heliceno\_5-chlor\_6-iod\_12-fluor\_13-iod\_15-fluor.log (2.2 SDs)
- 6Heliceno\_2-fluor\_6-iod\_7-chlor\_8-iod\_12-chlor.log (3.1 SDs)
- 6Heliceno\_3-chlor\_5-chlor\_8-fluor\_11-iod\_13-fluor\_16-iod.log (2.9 SDs)
- 6Heliceno\_2-chlor\_4-chlor\_7-fluor\_12-chlor.log (5.4 SDs)
- 6Heliceno\_2-iod\_15-iod\_16-iod.log (3.0 SDs)
- 6Heliceno\_3-iod\_5-fluor\_14-iod.log (4.4 SDs)
- 6Heliceno\_1-chlor\_2-iod\_3-iod.log (7.5 SDs)

Validation: 65 outliers out of 424 datapoints (15.3%)

- 6Heliceno\_8-brom\_11-brom\_12-brom\_13-iod\_14-chlor.log (5.6 SDs)
- 6Heliceno\_3-chlor\_10-chlor\_11-iod.log (2.8 SDs)
- 6Heliceno\_2-brom\_3-iod\_6-brom.log (2.6 SDs)
- 6Heliceno\_1-fluor\_5-chlor\_13-iod\_14-iod\_16-chlor.log (4.6 SDs)
- 6Heliceno\_4-iod\_5-iod\_11-chlor\_16-chlor.log (3.0 SDs)
- 6Heliceno\_3-iod\_5-iod\_7-brom\_9-iod\_10-fluor.log (2.6 SDs)
- 6Heliceno\_6-fluor\_7-chlor\_8-brom\_10-fluor\_12-iod\_15-brom.log (3.1 SDs)
- 6Heliceno\_2-chlor\_3-chlor\_5-chlor\_14-chlor\_15-chlor.log (4.1 SDs)
- 6Heliceno\_2-fluor\_3-chlor\_4-fluor\_9-chlor\_13-chlor.log (3.3 SDs)
- 6Heliceno\_1-fluor\_3-iod\_5-iod\_6-fluor\_15-iod\_16-iod.log (7.2 SDs)

Test: 49 outliers out of 235 datapoints (20.9%)

- 6Heliceno\_1-fluor\_4-brom\_5-brom\_9-iod\_11-fluor\_16-brom.log (4.0 SDs)
- 6Heliceno\_2-fluor\_5-fluor\_8-chlor\_12-iod\_13-brom\_16-fluor.log (6.1 SDs)
- 6Heliceno\_4-brom\_6-fluor\_8-brom\_12-iod\_13-iod\_15-fluor.log (6.6 SDs)
- 6Heliceno\_4-iod\_5-chlor\_8-fluor\_9-iod\_13-iod.log (3.8 SDs)
- 6Heliceno\_4-fluor\_5-fluor\_6-iod\_7-iod\_10-brom\_13-fluor.log (6.0 SDs)
- 6Heliceno\_10-iod\_11-iod\_14-iod.log (6.4 SDs)
- 6Heliceno\_6-fluor\_8-chlor\_11-chlor\_13-fluor\_15-iod\_16-iod.log (2.6 SDs)
- 6Heliceno\_1-brom\_3-fluor\_10-iod\_11-iod\_14-chlor.log

(4.3 SDs)

- 6Heliceno\_8-iod\_9-iod\_10-fluor\_13-iod.log (5.1 SDs)

- 6Heliceno\_1-iod\_2-chlor\_3-iod\_4-chlor\_12-chlor.log  
(2.1 SDs)

### PFI (only most important descriptors):

#### Outliers (max. 10 shown)

Train: 81 outliers out of 1907 datapoints (4.2%)

- 6Heliceno\_14-chlor\_15-iod.log (3.4 SDs)  
- 6Heliceno\_2-brom\_3-brom\_5-fluor\_12-fluor\_14-brom\_15  
-brom.log (2.3 SDs)

- 6Heliceno\_12-iod\_14-chlor\_16-brom.log (3.2 SDs)

- 6Heliceno\_5-chlor\_13-chlor.log (2.3 SDs)

- 6Heliceno\_2-fluor\_6-iod\_7-chlor\_8-iod\_12-chlor.log

(2.5 SDs)

- 6Heliceno\_2-iod\_15-iod\_16-iod.log (9.4 SDs)

- 6Heliceno\_3-iod\_5-fluor\_14-iod.log (3.3 SDs)

- 6Heliceno\_5-fluor\_11-iod\_12-iod\_13-fluor\_14-fluor\_1

6-brom.log (2.7 SDs)

- 6Heliceno\_1-chlor\_2-iod\_3-iod.log (3.4 SDs)

- 6Heliceno\_1-iod\_5-brom\_7-chlor\_8-chlor\_14-fluor\_15-

brom.log (2.1 SDs)

Validation: 49 outliers out of 212 datapoints (23.1%)

- 6Heliceno\_3-fluor\_7-brom\_14-chlor.log (2.3 SDs)

- 6Heliceno\_4-fluor\_16-chlor.log (2.1 SDs)

- 6Heliceno\_1-fluor\_5-chlor\_13-iod\_14-iod\_16-chlor.lo

g (4.7 SDs)

- 6Heliceno\_2-chlor\_13-iod.log (3.4 SDs)

- 6Heliceno\_4-fluor\_6-iod\_10-brom\_11-brom\_13-chlor\_15

-chlor.log (2.3 SDs)

- 6Heliceno\_7-chlor\_9-chlor\_15-fluor\_16-fluor.log (3.

1 SDs)

- 6Heliceno\_3-iod\_7-fluor\_8-brom\_9-brom\_15-brom.log (

3.3 SDs)

- 6Heliceno\_2-chlor\_3-brom\_7-iod\_9-chlor\_14-fluor\_15-

iod.log (3.2 SDs)

- 6Heliceno\_8-fluor\_11-fluor\_12-chlor\_14-chlor\_16-iod

.log (3.1 SDs)

- 6Heliceno\_1-fluor\_2-iod\_3-brom\_4-iod.log (4.5 SDs)

Test: 49 outliers out of 235 datapoints (20.9%)

- 6Heliceno\_1-fluor\_4-brom\_5-brom\_9-iod\_11-fluor\_16-b

rom.log (3.9 SDs)

- 6Heliceno\_2-fluor\_5-fluor\_8-chlor\_12-iod\_13-brom\_16

-fluor.log (4.9 SDs)

- 6Heliceno\_12-chlor\_13-iod\_16-fluor.log (2.1 SDs)

- 6Heliceno\_4-chlor\_6-iod\_8-chlor\_12-iod\_13-iod\_16-fl

uor.log (2.1 SDs)

- 6Heliceno\_4-fluor\_5-fluor\_6-iod\_7-iod\_10-brom\_13-fl

uor.log (4.2 SDs)

- 6Heliceno\_10-iod\_11-iod\_14-iod.log (2.9 SDs)

- 6Heliceno\_6-iod\_9-brom\_10-chlor\_11-iod\_12-brom\_14-b

rom.log (2.3 SDs)

- 6Heliceno\_6-fluor\_8-chlor\_11-chlor\_13-fluor\_15-iod\_

16-iod.log (7.9 SDs)

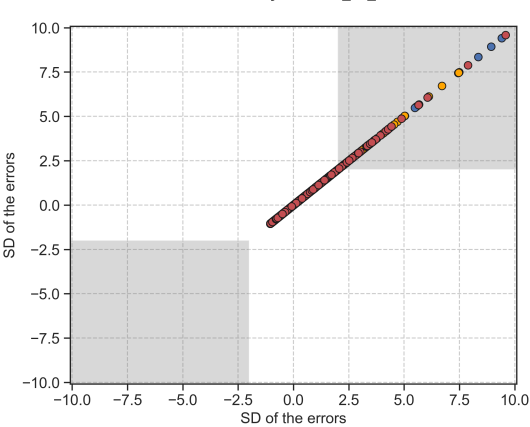
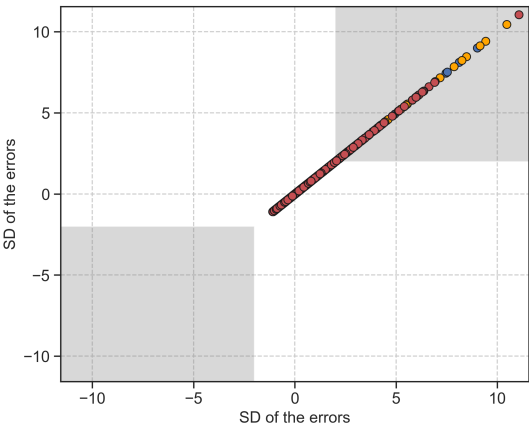
- 6Heliceno\_1-fluor\_4-brom\_5-fluor\_6-fluor\_8-fluor\_10

-iod.log (2.9 SDs)

- 6Heliceno\_1-brom\_3-fluor\_10-iod\_11-iod\_14-chlor.log

(6.1 SDs)

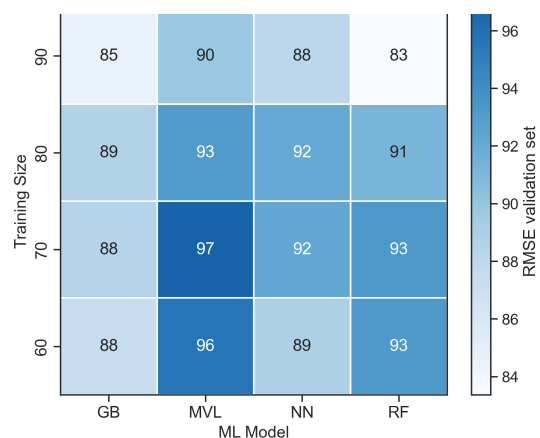
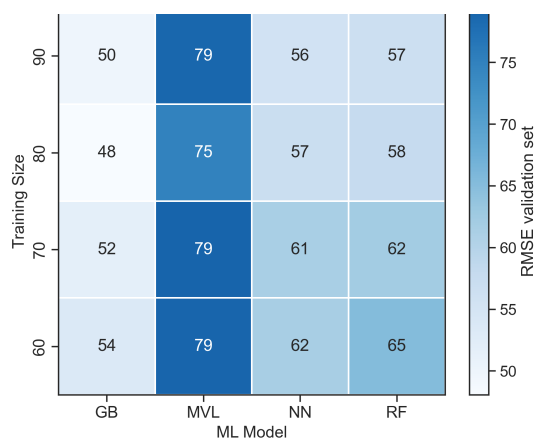






## Section F. Model Screening

This section compares different combinations of hyperoptimized algorithms and partition sizes.



## Section G. Reproducibility

This section provides all the instructions to reproduce the results presented.

### 1. Download these files (the authors should have uploaded the files as supporting information!):

- CSV database (reflected\_dataset\_Rmax.csv)

### 2. Install and adjust the versions of the following Python modules:

- Install ROBERT and its dependencies: `conda install -y -c conda-forge robert`
- Adjust ROBERT version: `pip install robert==1.2.1`
- Install scikit-learn-intelex: `pip install scikit-learn-intelex==2025.0.1`

(if scikit-learn-intelex is not installed, slightly different results might be obtained)

### 3. Run ROBERT using this command line in the folder with the CSV database:

```
python -m robert --ignore "[lambda,Nsubs,Names]" --names "Names" --y "R" --csv_name "reflected_dataset_Rmax.csv"
--corr_filter "False" --shap_show "16"
```

### 4. Execution time, Python version and OS:

Originally run in Python 3.10.16 using Windows 10.0.26100

Total execution time: 477.48 seconds (the number of processors should be specified by the user)



## Section H. Transparency

*This section contains important parameters used in scikit-learn models and ROBERT.*

### 1. Parameters of the scikit-learn models (same keywords as used in scikit-learn):

#### No PFI (standard descriptor filter):

sklearn model: GradientBoostingRegressor  
 random\_state: 19  
 names: Names  
 n\_estimators: 100  
 max\_depth: 5  
 max\_features: 0.5  
 min\_samples\_split: 2  
 min\_samples\_leaf: 1  
 min\_weight\_fraction\_leaf: 0  
 ccp\_alpha: 0  
 learning\_rate: 0.1  
 subsample: 1.0  
 validation\_fraction: 0.1

#### PFI (only most important descriptors):

sklearn model: RandomForestRegressor  
 random\_state: 43  
 names: Names  
 n\_estimators: 60  
 max\_depth: 100  
 max\_features: 0.75  
 min\_samples\_split: 2  
 min\_samples\_leaf: 1  
 min\_weight\_fraction\_leaf: 0  
 ccp\_alpha: 0  
 oob\_score: False  
 max\_samples: 0.75

### 2. ROBERT options for data split (KN or RND), predict type (REG or CLAS) and hyperopt error (RMSE, etc.):

#### No PFI (standard descriptor filter):

split: RND  
 type: reg  
 error\_type: rmse

#### PFI (only most important descriptors):

split: RND  
 type: reg  
 error\_type: rmse



## Section I. Abbreviations

*Reference section for the abbreviations used.*

|                                    |                                            |                                            |
|------------------------------------|--------------------------------------------|--------------------------------------------|
| <b>ACC:</b> accuracy               | <b>KN:</b> k-nearest neighbors             | <b>REG:</b> Regression                     |
| <b>ADAB:</b> AdaBoost              | <b>MAE:</b> root-mean-square error         | <b>RF:</b> random forest                   |
| <b>CSV:</b> comma separated values | <b>MCC:</b> Matthew's correl. coefficient  | <b>RMSE:</b> root mean square error        |
| <b>CLAS:</b> classification        | <b>ML:</b> machine learning                | <b>RND:</b> random                         |
| <b>CV:</b> cross-validation        | <b>MVL:</b> multivariate lineal models     | <b>SHAP:</b> Shapley additive explanations |
| <b>F1 score:</b> balanced F-score  | <b>NN:</b> neural network                  | <b>VR:</b> voting regressor                |
| <b>GB:</b> gradient boosting       | <b>PFI:</b> permutation feature importance |                                            |
| <b>GP:</b> gaussian process        | <b>R2:</b> coefficient of determination    |                                            |

## Miscellaneous

*General tips to improve the models and instructions to predict new values.*

### Some general tips to improve the score

1. Adding meaningful datapoints might help to improve the model. Also, using a uniform population of datapoints across the whole range of y values usually helps to obtain reliable predictions across the whole range. More information about the range of y values used is available in Section C.
2. Adding meaningful descriptors or replacing/deleting the least useful descriptors used might help. Feature importances are gathered in Section D.

### How to predict new values with these models?

1. Create a CSV database with the new points, including the necessary descriptors.
  2. Place the CSV file in the parent folder (i.e., where the module folders were created)
  3. Run the PREDICT module as 'python -m robert --predict --csv\_test FILENAME.csv'.
  4. The predictions will be shown at the end of the resulting PDF report and will be stored in the last column of two CSV files called MODEL\_SIZE\_test(\_No)\_PFI.csv, which are in the PREDICT folder.
-